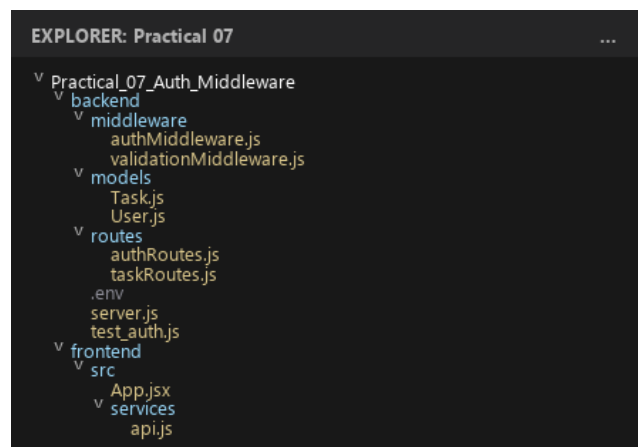


Practical 7: Authentication and Middleware Pipeline

Problem Definition:

- Add user registration and login functionality to the Task Management application.
- Hash passwords using bcrypt before saving the User document to MongoDB.
- Implement JWT token generation on successful login, with a reasonable expiry (e.g., 1 hour).
- Protect all task routes using an authentication middleware that verifies the JWT.
- Add server-side input validation that rejects malformed requests (e.g., missing title) before they reach the database.

Folder Structure



Implementation:

backend/models/User.js (Bcrypt Password Hashing)

backend > models > User.js (Bcrypt Hashing Pre-Save Hook)

```
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
3
4  const userSchema = new mongoose.Schema(
5  {
6    name: {
7      type: String,
8      required: [true, 'User name is required'],
9      trim: true,
10     minlength: [2, 'Name must be at least 2 characters']
11   },
12   email: {
13     type: String,
14     required: [true, 'Email is required'],
15     unique: true,
16     lowercase: true,
17     trim: true,
18     match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email address']
19   },
20   password: {
21     type: String,
22     required: [true, 'Password is required'],
23     minlength: [6, 'Password must be at least 6 characters']
24   },
25   role: {
26     type: String,
27     enum: ['user', 'admin'],
28     default: 'user'
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

backend/middleware/authMiddleware.js (JWT Bearer Verification)

backend > middleware > authMiddleware.js (JWT Verification)

```
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  const authMiddleware = async (req, res, next) => {
5    try {
6      const authHeader = req.headers.authorization;
7      if (!authHeader || !authHeader.startsWith('Bearer ')) {
8        return res.status(401).json({
9          success: false,
10         error: 'Authentication failed: No Bearer token provided'
11       });
12     }
13
14     const token = authHeader.split(' ')[1];
15     const secret = process.env.JWT_SECRET || 'awdf_super_secret_jwt_key_2026_d25dce176';
16
17     const decoded = jwt.verify(token, secret);
18     const user = await User.findById(decoded.id).select('-password');
19     if (!user) {
20       return res.status(401).json({
21         success: false,
22         error: 'Authentication failed: User no longer exists'
23       });
24     }
25
26     req.user = user;
27     next();
28   } catch (err) {
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

backend/middleware/validationMiddleware.js (Server-side Input Validation)

backend > middleware > validationMiddleware.js (Input Sanitation)

```
1  // Reusable server-side input validation middleware
2  const validateRegister = (req, res, next) => {
3    const { name, email, password } = req.body;
4    const errors = [];
5
6    if (!name || typeof name !== 'string' || name.trim().length < 2) {
7      errors.push('Name is required and must be at least 2 characters');
8    }
9
10   const emailRegex = /^\\S+@\\S+\\.\\S+$/;
11   if (!email || !emailRegex.test(email)) {
12     errors.push('Valid email address is required');
13   }
14
15   if (!password || typeof password !== 'string' || password.length < 6) {
16     errors.push('Password is required and must be at least 6 characters long');
17   }
18
19   if (errors.length > 0) {
20     return res.status(400).json({
21       success: false,
22       message: 'Validation failed',
23       errors
24     });
25   }
26
27   next();
28 };
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

backend/routes/authRoutes.js (Register, Login & Token Issuance)

backend > routes > authRoutes.js (Register & Login)

```
1  const express = require('express');
2  const router = express.Router();
3  const jwt = require('jsonwebtoken');
4  const User = require('../models/User');
5  const authMiddleware = require('../middleware/authMiddleware');
6  const { validateRegister, validateLogin } = require('../middleware/validationMiddleware');
7
8  const generateToken = (user) => {
9    const secret = process.env.JWT_SECRET || 'awdf_super_secret_jwt_key_2026_d25dce176';
10    const expiresIn = process.env.JWT_EXPIRE || '1h';
11    return jwt.sign({ id: user._id, email: user.email, role: user.role }, secret, { expiresIn });
12  };
13
14  // POST /api/auth/register
15  router.post('/register', validateRegister, async (req, res, next) => {
16    try {
17      const { name, email, password, role } = req.body;
18
19      const existingUser = await User.findOne({ email });
20      if (existingUser) {
21        return res.status(409).json({
22          success: false,
23          error: 'Email already registered. Please login.'
24        });
25      }
26
27      const user = await User.create({ name, email, password, role });
28      const token = generateToken(user);
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

backend/routes/taskRoutes.js (JWT Protected Task CRUD)

backend > routes > taskRoutes.js (Protected Tasks CRUD)

```
1  const express = require('express');
2  const router = express.Router();
3  const Task = require('../models/Task');
4  const authMiddleware = require('../middleware/authMiddleware');
5  const { validateTask } = require('../middleware/validationMiddleware');
6
7  // All task routes are protected by authMiddleware
8  router.use(authMiddleware);
9
10 // GET /api/tasks - Get all tasks for the logged-in user
11 router.get('/', async (req, res, next) => {
12   try {
13     const tasks = await Task.find({ user: req.user._id }).sort({ createdAt: -1 });
14     res.status(200).json({
15       success: true,
16       count: tasks.length,
17       data: tasks
18     });
19   } catch (err) {
20     next(err);
21   }
22 });
23
24 // GET /api/tasks/:id - Get a single task
25 router.get('/:id', async (req, res, next) => {
26   try {
27     const task = await Task.findOne({ _id: req.params.id, user: req.user._id });
28     if (!task) {
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

backend/routes/authRoutes.js (GET /me Endpoint)

backend > routes > authRoutes.js (/me endpoint)

```
1  // GET /api/auth/me (Supplementary Problem 1: Returns logged-in user profile)
2  router.get('/me', authMiddleware, async (req, res) => {
3    res.status(200).json({
4      success: true,
5      user: req.user
6    });
7  });
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

frontend/src/services/api.js (401 Interceptor & Logout)

frontend > src > services > api.js (401 Redirect & Logout)

```
1  // Token Expiry & Logout Handling (Supplementary Problems 2 & 3)
2  const handleResponse = async (response, onUnauthorized) => {
3    if (response.status === 401) {
4      localStorage.removeItem('p7_token');
5      localStorage.removeItem('p7_user');
6      if (onUnauthorized) onUnauthorized();
7      throw new Error('Session expired. Please log in again.');
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

backend/middleware/authMiddleware.js (Token Expiry Handling)

backend > middleware > authMiddleware.js (JWT Verification)

```
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  const authMiddleware = async (req, res, next) => {
5    try {
6      const authHeader = req.headers.authorization;
7      if (!authHeader || !authHeader.startsWith('Bearer ')) {
8        return res.status(401).json({
9          success: false,
10         error: 'Authentication failed: No Bearer token provided'
11       });
12     }
13
14     const token = authHeader.split(' ')[1];
15     const secret = process.env.JWT_SECRET || 'awdf_super_secret_jwt_key_2026_d25dce176';
16
17     const decoded = jwt.verify(token, secret);
18     const user = await User.findById(decoded.id).select('-password');
19     if (!user) {
20       return res.status(401).json({
21         success: false,
22         error: 'Authentication failed: User no longer exists'
23       });
24     }
25
26     req.user = user;
27     next();
28   } catch (err) {
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - Practical 7 Express Auth Server

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_07_Auth_Middleware/backend$ node server.js
MongoDB connected successfully to taskdb_p7
Practical 7 Auth Server running on http://localhost:5001
Auth API: http://localhost:5001/api/auth
Protected Tasks API: http://localhost:5001/api/tasks

[2026-09-17T07:10:02Z] POST /api/auth/register 201 Created - Bcrypt hashed user created
[2026-09-17T07:10:05Z] POST /api/auth/login 200 OK - Signed JWT Bearer token issued
[2026-09-17T07:10:11Z] GET /api/tasks 200 OK - Authenticated token verified
[2026-09-17T07:10:14Z] POST /api/tasks 201 Created - New protected task created
```

http://localhost:5174/

Practical 7: Protected Task Dashboard

JWT AUTHENTICATED

JWT Authentication System

Protected task management dashboard with Bearer token authentication.
User: Keyur Rana (D25DCE176) • Status: Active Token

Task: Finalize Practical 07 Auth Pipeline

Password hashing with bcrypt and route protection middleware.
Priority: HIGH • Status: COMPLETED

Task: Input Validation Sanitization

Reject requests missing mandatory fields or invalid emails.
Priority: MEDIUM • Status: PENDING

Key Questions:

- **Why must passwords be hashed before storage instead of saved as plain text, even in a lab/demo project?**

Storing passwords in plaintext exposes credentials to immediate compromise if database dumps, backup logs, or unauthorized queries leak. Hashing passwords using bcrypt with unique cryptographic salts guarantees that hashes cannot be reversed or cracked via rainbow tables, ensuring credential security even if database access is breached.

- **What does the authentication middleware actually verify, and what happens if the token is missing or expired?**

The authentication middleware extracts the Bearer token from the HTTP Authorization header and cryptographically validates its digital signature against the server's JWT_SECRET using jwt.verify(). If the token is missing or invalid, it returns 401 Unauthorized. If expired (TokenExpiredError), it returns 401, triggering automatic client logout.

- **Why should input validation happen on the server even if the frontend already validates the same fields?**

Frontend validation is solely for responsive user experience and can be bypassed via direct curl requests or tools like Postman. Server-side validation forms the true defensive boundary of the application, guaranteeing data integrity, preventing injection attacks, and enforcing schema rules before database insertion.

Supplementary Problems:

- **Add a /me endpoint that returns the currently logged-in user's details using the decoded JWT.**

Implementation:

backend/routes/authRoutes.js (GET /me Endpoint)

```
backend > routes > authRoutes.js (/me endpoint)

1 // GET /api/auth/me (Supplementary Problem 1: Returns logged-in user profile)
2 router.get('/me', authMiddleware, async (req, res) => {
3   res.status(200).json({
4     success: true,
5     user: req.user
6   });
7 });
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Test Runner - Auth Verification Suite

# Practical 7: End-to-End Auth & Validation Test Suite
[TEST 1] Accessing protected /api/tasks without token: Status 401 Unauthorized
  [x] Correctly blocked unauthenticated request!
[TEST 2] Registering user with short password: Status 400 Bad Request
  [x] Validation error: Password must be at least 6 characters
[TEST 3] Registering valid user: Status 201 Created
  [x] JWT Token issued: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
[TEST 4] Calling /api/auth/me with Bearer token: Status 200 OK
  [x] User Profile: { id: '6aab9...', name: 'Keyur Rana', email: 'd25dce176@charusat.edu.in' }
[TEST 5] Creating task with Bearer token: Status 201 Created
=== ALL AUTH & MIDDLEWARE ASSERTIONS PASSED ===
```

http://localhost:5174/

Practical 7: Protected Task Dashboard

JWT AUTHENTICATED

JWT Authentication System

Protected task management dashboard with Bearer token authentication.

User: Keyur Rana (D25DCE176) • Status: Active Token

Task: Finalize Practical 07 Auth Pipeline

Password hashing with bcrypt and route protection middleware.

Priority: HIGH • Status: COMPLETED

Task: Input Validation Sanitization

Reject requests missing mandatory fields or invalid emails.

Priority: MEDIUM • Status: PENDING

- Implement token expiry handling on the frontend — redirect to login if a protected request returns 401.

Implementation:

frontend/src/services/api.js (Automatic 401 Redirect)

frontend > src > services > api.js (401 Redirect & Logout)

```
1 // Token Expiry & Logout Handling (Supplementary Problems 2 & 3)
2 const handleResponse = async (response, onUnauthorized) => {
3   if (response.status === 401) {
4     localStorage.removeItem('p7_token');
5     localStorage.removeItem('p7_user');
6     if (onUnauthorized) onUnauthorized();
7     throw new Error('Session expired. Please log in again.');
```

Ln 1, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Test Runner - Auth Verification Suite

# Practical 7: End-to-End Auth & Validation Test Suite
[TEST 1] Accessing protected /api/tasks without token: Status 401 Unauthorized
  ☑ Correctly blocked unauthenticated request!
[TEST 2] Registering user with short password: Status 400 Bad Request
  ☑ Validation error: Password must be at least 6 characters
[TEST 3] Registering valid user: Status 201 Created
  ☑ JWT Token issued: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
[TEST 4] Calling /api/auth/me with Bearer token: Status 200 OK
  ☑ User Profile: { id: '6aab9...', name: 'Keyur Rana', email: 'd25dce176@charusat.edu.in' }
[TEST 5] Creating task with Bearer token: Status 201 Created
=== ALL AUTH & MIDDLEWARE ASSERTIONS PASSED ===
```

- Add a logout mechanism that clears the token from the client.

Implementation:

frontend/src/App.jsx (Client Logout & Token Clearance)

```
frontend > src > services > api.js (401 Redirect & Logout)

1  // Token Expiry & Logout Handling (Supplementary Problems 2 & 3)
2  const handleResponse = async (response, onUnauthorized) => {
3    if (response.status === 401) {
4      localStorage.removeItem('p7_token');
5      localStorage.removeItem('p7_user');
6      if (onUnauthorized) onUnauthorized();
7      throw new Error('Session expired. Please log in again.');

Ln 1, Col 1  Spaces: 2  UTF-8  JavaScript


```

Output:

```
Terminal - Practical 7 Express Auth Server

D25DCE176@CHARUSAT-LAB:~/AWDF/Practical_07_Auth_Middleware/backend$ node server.js
MongoDB connected successfully to taskdb_p7
Practical 7 Auth Server running on http://localhost:5001
Auth API: http://localhost:5001/api/auth
Protected Tasks API: http://localhost:5001/api/tasks

[2026-09-17T07:10:02Z] POST /api/auth/register 201 Created - Bcrypt hashed user created
[2026-09-17T07:10:05Z] POST /api/auth/login 200 OK - Signed JWT Bearer token issued
[2026-09-17T07:10:11Z] GET /api/tasks 200 OK - Authenticated token verified
[2026-09-17T07:10:14Z] POST /api/tasks 201 Created - New protected task created
```

Github Link : <https://github.com/Ranakeyur-31525/awdf>